

NAANMUDHALVAN

Project Report

Implement Client Script & UI Policy (Incident)

Enforcing data integrity on ServiceNow Incident records

Team ID	SWTID-2026-4934
Team Size	5
Team Leader	Rashmi P P
Team Members	Ravichandran Rishikesh MB Rohith Srinivasan Rubika Sakthi M
Platform	ServiceNow
Program	Naan Mudhalvan, through SmartBridge (MySkillWallet)

Project documentation prepared by Rohith Srinivasan

1. INTRODUCTION

1.1 Project Overview

Incident records are where most service desk work begins. They are triaged, routed to the right group and measured against SLAs, so the quality of the information entered when an incident is logged has a direct effect on how quickly and how correctly it gets resolved. When that quality depends only on people remembering to fill in the right things, gaps and inconsistencies are bound to appear.

This project, Implement Client Script & UI Policy (Incident), shows how ServiceNow's client-side controls can protect data quality at the moment of entry. Rather than trusting manual checks, we configured the Incident form so that it responds to the impact of the incident, fills in values where that makes sense, and refuses to save a record that breaks a business rule.

The solution is made up of six connected pieces of work:

- **UI Policy "High Impact Control"**: applies to the Incident table when Impact is 1 - High, makes Assignment group mandatory, and reverses itself automatically when the condition no longer holds.
- **UI Policy Action for Urgency**: makes the Urgency field read-only while the policy condition is met.
- **onChange Client Script**: sets Urgency to High as soon as Impact is changed to High.
- **onSubmit Client Script**: stops a High impact incident from being saved when Assigned to is empty.
- **onCellEdit Client Script**: prevents the State field from being changed directly in the list view.
- **Structured testing**: covering mandatory enforcement, successful save, the reverse condition, list-edit blocking and form-based updates.

Skills covered: Incident Management, UI Policy, UI Policy Actions, Client Scripts and Form Validation.

1.2 Purpose

The purpose of this project is to demonstrate, through a working configuration, how data integrity can be enforced directly on the Incident form. Specifically, the project aims to:

- Make fields mandatory only when a defined condition is true, and return them to normal when it is not.
- Auto-populate values so that related fields, such as Impact and Urgency, stay consistent.
- Control field behaviour, for example by locking a field as read-only.
- Prevent record submission when required conditions are not met, and tell the user why.
- Improve the quality and consistency of Incident data, which in turn supports reporting accuracy, SLA compliance and overall service quality.

Because the scope is deliberately small and focused, the project also works as a compact, practical introduction to the way UI Policies and Client Scripts complement each other.

2. IDEATION PHASE

2.1 Problem Statement

Incident records need consistent and accurate data to be triaged, routed and resolved effectively. In practice, relying only on user awareness and manual checks means that incomplete, inconsistent or incorrect data can still be submitted. This affects reporting accuracy, SLA compliance and the overall quality of service. What is needed is a way to enforce conditional field behaviour and validation at the user interface level, so that an Incident meets the defined business rules before it is submitted.

Aspect	Description
Who is affected	Users who create and update Incident records, and the teams who depend on that data for triage, routing and reporting.

Aspect	Description
What is needed	Incident records that are complete, consistent and compliant with business rules at the time they are saved.
The problem	Manual checks and user awareness alone allow incomplete, inconsistent or incorrect data to be submitted.
Impact of the problem	Reduced reporting accuracy, weaker SLA compliance and lower service quality.
Approach taken	Use UI Policies and Client Scripts to control field visibility, mandatory status and save-time validation on the Incident form.

2.2 Empathy Map Canvas

The empathy map below looks at the situation through the eyes of a person who logs or updates incidents. It is drawn from the problem described in the project brief rather than from a formal survey, and it helped us decide which behaviours the form should guide or restrict.

SAYS "Which fields do I have to fill in for a high-impact incident?" "Why won't this record save?"	THINKS The incident is filled in well enough for now. Details such as assignment can be sorted out later.
DOES Logs incidents quickly and may leave key fields empty. Changes fields directly, including State from the list view.	FEELS Under time pressure and unsure which rules apply. Frustrated when a problem is discovered late or without explanation.
PAINS Incomplete, inconsistent or incorrect submissions. Rework, SLA risk and unreliable reports.	GAINS Clear, immediate guidance on the form itself. Fields that fill in or lock automatically, and problems caught before saving.

2.3 Brainstorming

We considered several ways to keep Incident data clean and weighed each against the problem statement. The table records the ideas and what we decided.

#	Idea	What it would achieve	Decision
1	Rely on user awareness and manual checks	Nothing new; this is the situation the brief identifies as the cause of the problem.	Not chosen
2	UI Policy that makes Assignment group mandatory when Impact is High	Conditional mandatory field without any code.	Selected
3	UI Policy Action that makes Urgency read-only	Stops users overriding Urgency on high-impact incidents.	Selected
4	onChange script that sets Urgency when Impact becomes High	Automatic population so Impact and Urgency stay aligned.	Selected
5	onSubmit script that blocks saving without Assigned to	Save-time validation with a clear error message.	Selected
6	onCellEdit script that blocks State changes in the list	Protects State from quick edits that bypass the form.	Selected
7	Enable "Reverse if false" on the UI Policy	Restores normal behaviour automatically when Impact is no longer High.	Selected

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey below follows a user from opening a new Incident to updating it later, and shows what the system does at each stage.

Stage	What the user does	What the system does	Result
1. Start	Opens Incident → Create New.	Loads a new Incident form and evaluates the UI Policy on load.	Form is ready.
2. Set impact	Selects Impact as 1 - High.	UI Policy applies: Assignment group becomes mandatory and Urgency becomes read-only. The onChange script sets Urgency to 1 - High.	Form guides the user.
3. Complete the form	Fills in Caller, Short description and Assignment group.	Mandatory fields are marked on the form.	Required data is captured.
4. Submit without owner	Clicks Submit with Assigned to empty.	onSubmit script shows an error under Assigned to and cancels the save.	Record is not saved.
5. Correct and save	Selects a user in Assigned to and submits again.	Validation passes.	Record is saved.
6. Lower the impact	Changes Impact from High to Medium.	Reverse if false undoes the policy actions; Urgency becomes editable and Assigned to is no longer enforced.	Record saves normally.
7. Edit from the list	Double-clicks State in the incident list.	onCellEdit script shows an alert and cancels the edit.	State is unchanged.
8. Edit from the form	Opens the incident and changes State on the form.	Change is allowed.	State is updated.

3.2 Solution Requirements

Functional requirements

ID	Requirement
FR1	Create an active UI Policy named High Impact Control on the Incident table, triggered when Impact is 1 - High.
FR2	Make Assignment group mandatory while the condition is met, and reverse this automatically when it is not (Reverse if false).
FR3	Make Urgency read-only while the condition is met, leaving its visibility unchanged.
FR4	When Impact is changed to High, set Urgency to 1 - High automatically. The script must not run during form load or when the new value is empty.
FR5	Block saving when Impact is High and Assigned to is empty, and show an error message against the Assigned to field.
FR6	Block edits to the State field made directly in the list view, show an alert and keep the original value.
FR7	Continue to allow State to be changed from the Incident form.

Non-functional and environment requirements

- A ServiceNow instance and a user with administrative or configuration access.
- Lightweight and efficient: each rule is a small configuration or a short script.
- Immediate feedback: users see the result of a rule on the form itself, without waiting for a server response or a later review.
- Reversible: rules that depend on a condition must switch off cleanly when the condition changes.

- Easy to implement and maintain, making it suitable for a short-duration micro project.

3.3 Data Flow Diagram

The diagram shows how user actions on the Incident form and list flow into the different client-side controls, and what each control returns. The UI Policy and the onChange script react as the user changes Impact, the onSubmit script runs when the user tries to save, and the onCellEdit script runs when a value is edited inline in a list.

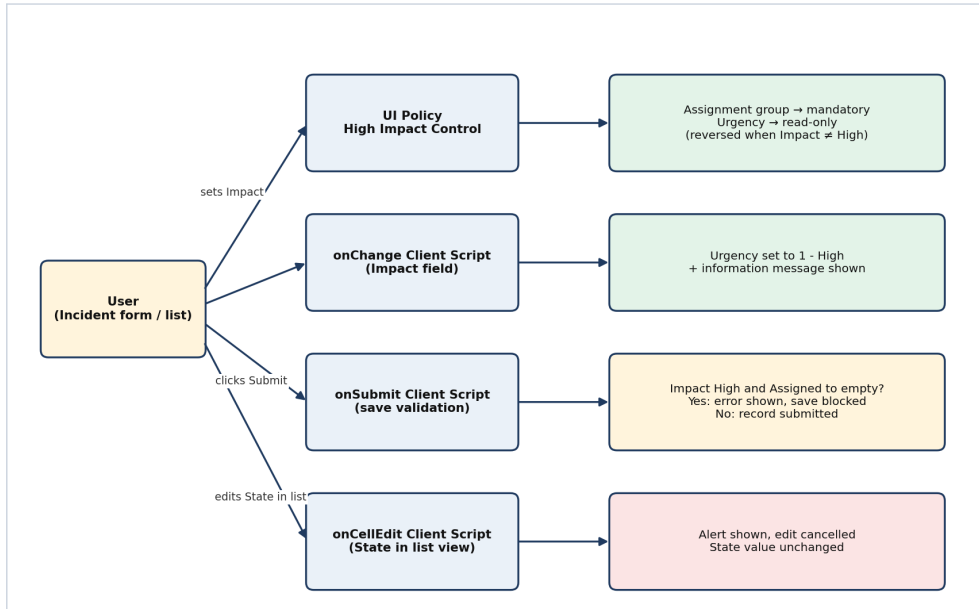


Figure 1: Data flow between user actions, client-side controls and their outcomes.

3.4 Technology Stack

Area	Technology / Component
Platform	ServiceNow (developer instance used for configuration and testing)
Application module	Incident Management, using the Incident [incident] table
Declarative controls	UI Policy and UI Policy Actions (System UI → UI Policies)
Scripted controls	Client Scripts of type onChange, onSubmit and onCellEdit (UI Type: Desktop)
Scripting language and API	JavaScript with the GlideForm API (g_form): setValue, getValue, addInfoMessage / showFieldMsg and showErrorBox
Access level	Administrator or configuration access

4. PROJECT DESIGN

4.1 Problem Solution Fit

Each part of the problem has a matching control in the solution, and each control is placed where the user actually works, on the form or in the list.

Problem observed	Solution component
High impact incidents are submitted without routing information.	UI Policy makes Assignment group mandatory; onSubmit script requires Assigned to.
Urgency does not match a high Impact value.	onChange script sets Urgency to High; UI Policy Action locks the field as read-only.
Incorrect or incomplete records reach the database.	onSubmit script cancels the save and displays an error message.

Problem observed	Solution component
State is changed quickly from the list without opening the record.	onCellEdit script blocks the change and asks the user to open the Incident.
Rules should not stay in force after the condition changes.	"Reverse if false" restores the fields when Impact is no longer High.

4.2 Proposed Solution

The proposed solution combines one UI Policy with two actions and three Client Scripts. The UI Policy handles behaviour that can be described with a simple condition. The Client Scripts handle the cases that need logic: setting a value, validating at save time, and intercepting a list edit.

Component	Type / trigger	Behaviour
High Impact Control	UI Policy, Incident table, condition Impact is 1 - High	Applies its actions while Impact is High; Reverse if false is enabled.
Action: Assignment group	UI Policy Action	Mandatory: True. Read only: False.
Action: Urgency	UI Policy Action	Read only: True. Visible and Mandatory left alone.
Auto set urgency for high impact	Client Script, onChange on Impact	Sets Urgency to 1 when Impact becomes 1 and informs the user.
Prevent save if assigned to missing	Client Script, onSubmit	Returns false and shows an error on Assigned to when Impact is 1 and Assigned to is empty.
Prevent state change via list edit	Client Script, onCellEdit on State	Shows an alert and cancels the list edit.

Key configuration details of the UI Policy are summarised below.

Setting	Value
Short description (name)	High Impact Control
Table	Incident [incident]
Application	Global
Active	True
Order	100
Condition	Impact is 1 - High
Global / On load	Selected / Selected
Reverse if false	Selected
Inherit	Not selected

4.3 Solution Architecture

The architecture has three layers. Users interact with the Incident form and list in the browser. Between them and the database sits the client-side control layer, made up of the UI Policy and the three Client Scripts. Only data that passes these checks is submitted to the Incident table.

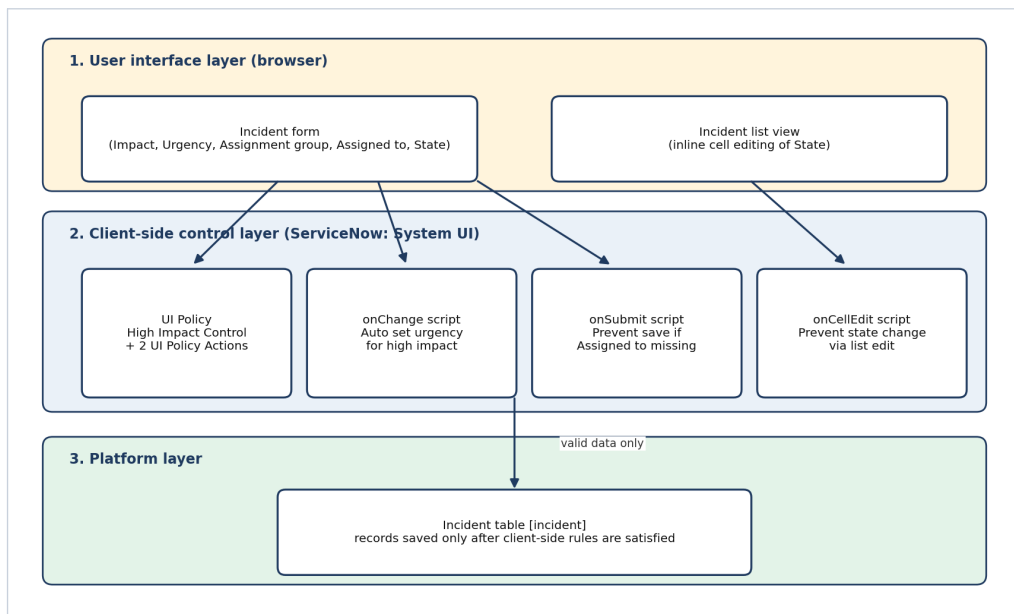


Figure 2: Layered solution architecture for the Incident client-side controls.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was organised as six tasks, each building on the one before it. Since it is a micro project, the work was kept short and focused, with configuration first and testing at the end.

Task	Activity	Work carried out	Status
Task 1	Create UI Policy on Incident	Created High Impact Control with the condition Impact is 1 - High, added the Assignment group action (mandatory) and enabled Reverse if false.	Completed
Task 2	Create UI Policy Action: Urgency	Added an action on the policy to make Urgency read-only, leaving Visible unchanged.	Completed
Task 3	Create onChange Client Script	Wrote and activated "Auto set urgency for high impact" on the Impact field.	Completed
Task 4	Create onSubmit Client Script	Wrote and activated "Prevent save if assigned to missing" for save-time validation.	Completed
Task 5	Create onCellEdit Client Script	Wrote and activated "Prevent state change via list edit" on the State field.	Completed
Task 6	Test the configuration	Ran five test activities covering mandatory enforcement, successful save, reverse condition, list-edit blocking and form-based update.	Completed

6. FUNCTIONAL AND PERFORMANCE TESTING

Testing followed the five activities defined for the project. Each test was run on the ServiceNow instance and the observed result was compared with the expected result.

Test	Scenario and steps	Expected result	Observed result	Status
TC-01	Mandatory enforcement. Create a new Incident, set Impact to High, leave Assigned to empty and click Submit.	Record is not saved; an error message appears for Assigned to.	Save was blocked and the message "Assigned To is mandatory for High impact incidents." appeared under Assigned to. Urgency showed 1 - High as read-only and Assignment group was marked mandatory.	Pass
TC-02	Successful save. Select a user in Assigned to and click Submit.	Record is saved without errors.	With Assigned to filled in (Beth Anglin), no error was raised and the incident INC0010012 was created and appeared in the Incident list.	Pass
TC-03	Reverse condition. Open an incident with High impact and change Impact to Medium.	Assigned to no longer enforced; Urgency editable; record saves.	With Impact at 2 - Medium, Assigned to could be left empty and Urgency was editable again.	Pass
TC-04	List edit blocking. In the Incident list, double-click the State field of a record.	Alert appears and State is unchanged.	An alert reading "State cannot be changed from the list. Please open the Incident." appeared and the edit was cancelled.	Pass
TC-05	Form-based update. Open the incident, change State and click Update.	State change is saved.	INC0008111 was updated to In Progress from the form and the list showed the new State.	Pass

6.1 Performance Testing

This project focused on functional correctness, so no formal load or response-time measurements were taken. From a design point of view, the controls are light on resources: the UI Policy is a simple declarative rule, and each Client Script is only a few lines long and runs in the user's browser as the form or list is used. In all the test runs above, the form and list responded immediately to each action.

7. RESULTS

7.1 Output Screenshots

The screenshots below record the implementation and testing, from the UI Policy configuration to confirming the behaviour on the form and in the list.

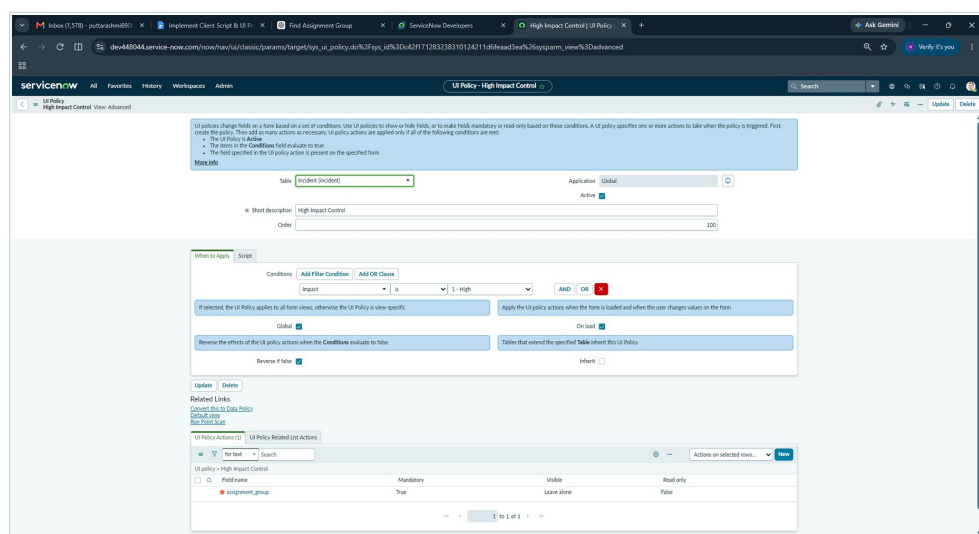


Figure 3: High Impact Control UI Policy on the Incident table with the condition Impact is 1 - High, Reverse if false enabled, and the Assignment group action (Mandatory: True).

UI Policy Action configuration for Urgency:

- UI policy:** High Impact Control
- Table:** Incident [incident]
- * Field name:** Urgency
- Application:** Global
- Mandatory:** Leave alone
- Visible:** Leave alone
- Read only:** True
- Clear the field value:** ☐

Buttons: Update, Delete

Figure 4: UI Policy Action for Urgency with Read only set to True.

Client Script configuration for "Auto set urgency for high impact":

- Name:** Auto set urgency for high impact
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onChange
- Field name:** Impact
- Script:**

```

function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue === '') {
        return;
    }
    if (newValue == "1") {
        $form.setFieldValue("urgency", "High");
        $form.setFieldValue("urgency", "High");
    }
}

```
- Update Versions:**

Name	Recorded at	State	Source	Reverted from
inc_script_client_4a8f93f9a3b330124211d8aadf4a	2024-09-27 23:50:39	Current	Update Set Default	(empty)

Buttons: Update, Delete, Run Default Script

Figure 5: onChange Client Script "Auto set urgency for high impact" on the Impact field.

Client Script configuration for "Prevent save if Assigned to missing":

- Name:** Prevent save if Assigned to missing
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onSubmit
- Script:**

```

function onSubmit() {
    if ($form.getFieldValue("Impact") == "1" &&
        $form.getFieldValue("assigned_to") == "") {
        $form.showErrorBox(
            "assigned to",
            "assigned to is mandatory for High impact incidents."
        );
        return false;
    }
    return true;
}

```

Buttons: Update, Delete, Run Default Script

Figure 6: onSubmit Client Script "Prevent save if assigned to missing".

The screenshot shows the 'Client Script - New Record' form in ServiceNow. The script is named 'Prevent state change via list edit' and is associated with the 'Incident' table. The UI type is 'Desktop' and the type is 'onCellEdit'. The field name is 'State'. The script code is as follows:

```

1 function onCellEdit(system, table, oldValue, newValue, callback) {
2   alert('State cannot be updated using list editing. Please open the Incident.');
```

```

3   callback(false);
4 }
5
6
7

```

The 'Isolate script' checkbox is checked. The 'Submit' button is visible at the bottom.

Figure 7: onCellEdit Client Script "Prevent state change via list edit" on the State field.

The screenshot shows the 'Incident - Create INC0010002' form. The 'Urgency' is set to 'High'. The 'Impact' is set to 'High'. The 'Urgency' field is read-only. The 'Assignment group' is set to 'Software'. The 'Assigned to' field is empty. The 'Short description' is 'Testing UI policy'. The 'Description' field is empty. The 'Related Search' results are displayed at the bottom.

Errors and warnings:

- Urgency set to High for High Impact Incident.
- The following mandatory fields are not filled in: Short description, Assignment group
- Assigned To is mandatory for High Impact incidents.

Figure 8: Impact set to High: Urgency is read-only, Assignment group is mandatory and the save is blocked with an error under Assigned to.

The screenshot shows the 'Incident - Create INC0010002' form. The 'Urgency' is set to 'High'. The 'Impact' is set to 'High'. The 'Urgency' field is read-only. The 'Assignment group' is set to 'Software'. The 'Assigned to' field is now filled with the user 'Beth Anglin'. The 'Short description' is 'Testing UI policy'. The 'Description' field is empty. The 'Related Search' results are displayed at the bottom.

Errors and warnings:

- Urgency set to High for High Impact Incident.
- The following mandatory fields are not filled in: Short description, Assignment group

Figure 9: Assigned to filled in with a user, so the incident can be submitted successfully.

Figure 10: Reverse condition test: Impact changed to 2 - Medium, Urgency editable and Assigned to no longer enforced.

Number	Opened	Short description	User	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010002	2026-09-28 03:11:25	Testing UI policy	Adela Cervantiz	2 - High	In Progress	Inquiry / Help	Software	(empty)	2026-09-28 03:27:30	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	1 - Critical	New	Inquiry / Help	Application Development	system	2026-09-28 02:03:22	system
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	Resolved	(empty)	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	(empty)	(empty)	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment - ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF: Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
INC0008001	2021-01-15 13:04:14	ATF: TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-06-12 15:15:11	system
INC0001990	2020-06-07 09:02:25	Unable to access the personal details section in payroll portal	Problem CoordinatorATF	5 - Planning	On Hold	Inquiry / Help	(empty)	(empty)	2020-06-07 09:04:34	admin
INC0000599	2016-08-10 09:14:29	Unable to access team file share	Rick Bertie	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:29	admin
INC0000588	2016-08-10 09:37:45	Performance problems with email	Bow Ruggeri	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:37:45	admin
INC0000557	2016-08-10 09:14:59	Performance problems with wifi	Bertie Luby	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:59	admin
INC0000555	2026-05-21 21:47:23	SAP Sales app is not accessible	Carol Coughlin	1 - Critical	In Progress	(empty)	Service Desk	Beth Anglin	2026-06-04 16:08:31	admin
INC0000554	2015-11-02 12:49:08	SAP Materials Management is slow or there is an outage	Christen Mitchell	1 - Critical	On Hold	Software	Service Desk	(empty)	2015-11-24 07:47:36	admin
INC0000553	2026-05-21 13:48:46	The SAP HR application is not accessible	Margaret Grey	1 - Critical	In Progress	Inquiry / Help	Software	Beth Anglin	2026-06-11 12:42:25	admin
INC0000552	2026-05-21 13:48:40	SAP Financial Accounting application appears to be down	Bud Richman	1 - Critical	In Progress	Software	Software	Fred Luddy	2026-06-04 14:50:24	system
INC0000551	2026-05-21 13:48:32	Manager can't access SAP Controlling application	Joe Employee	1 - Critical	In Progress	Software	Software	Don Goodliffe	2026-06-11 12:40:45	admin
INC0000550	2026-05-21 14:58:24	Can't access Exchange server - Is it down?	Jerrod Bennett	1 - Critical	In Progress	Hardware	Hardware	Beth Anglin	2026-06-11 12:49:40	admin
INC000049	2026-05-21 14:56:37	Network storage unavailable	Beth Anglin	2 - High	In Progress	Network	Hardware	Don Goodliffe	2026-06-11 12:52:34	admin

Figure 11: Alert shown when State is edited from the list, confirming the change is blocked.

Figure 12: State changed to In Progress from the Incident form, which is allowed.

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010002	2024-09-28 03:11:25	Testing UI policy	Adela Cervantoz	2 - High	In Progress	Software	(empty)	(empty)	2024-09-28 03:27:30	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	1 - Critical	New	Inquiry / Help	Application Development	(empty)	2018-09-28 02:03:22	system
INC0009005	2018-08-31 21:55:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment - ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF - Test1	System Administrator	4 - Low	In Progress	Inquiry / Help	(empty)	(empty)	2024-09-28 03:32:38	admin
INC0008001	2021-01-15 13:04:14	ATF TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Opspace	(empty)	2024-06-12 15:15:11	system
INC0001990	2020-06-07 09:02:25	Unable to access the personal details section in payroll portal	Problem CoordinatorATF	5 - Planning	On Hold	Inquiry / Help	(empty)	(empty)	2020-06-07 09:04:34	admin
INC0000559	2016-08-10 09:14:29	Unable to access team file share	Rick Berle	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:29	admin
INC0000558	2016-08-10 09:37:45	Performance problems with email	Bow Ruggeri	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:37:45	admin
INC0000557	2016-08-10 09:14:59	Performance problems with wifi	Bertie Luby	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2016-08-10 09:14:59	admin
INC0000555	2024-05-21 21:47:23	SAP Sales app is not accessible	Carol Coughlin	1 - Critical	In Progress	Service Desk	Service Desk	Beth Anglin	2024-06-04 16:08:31	admin
INC0000554	2015-11-02 12:49:08	SAP Materials Management is slow or there is an outage	Christen Mitchell	1 - Critical	On Hold	Software	Service Desk	(empty)	2015-11-24 07:47:36	admin
INC0000553	2024-05-21 13:48:46	The SAP HR application is not accessible	Margaret Grey	1 - Critical	In Progress	Inquiry / Help	Software	Beth Anglin	2024-06-11 12:40:25	admin
INC0000552	2024-05-21 13:48:40	SAP Financial Accounting application appears to be down	Bud Richman	1 - Critical	In Progress	Software	Software	Fred Luddy	2024-06-04 14:50:24	system
INC0000551	2024-05-21 13:48:32	Manager can't access SAP Controlling application	Joe Employee	1 - Critical	In Progress	Software	Software	Don Goodlife	2024-06-11 12:40:45	admin
INC0000550	2024-05-21 14:58:24	Can't access Exchange server - is it down?	Jerrold Bennett	1 - Critical	In Progress	Hardware	Hardware	Beth Anglin	2024-06-11 12:49:40	admin
INC0000449	2024-05-21 14:56:37	Network storage unavailable	Beth Anglin	2 - High	In Progress	Network	Hardware	Don Goodlife	2024-06-11 12:52:34	admin

Figure 13: Incident list showing INC0008111 with the updated State of In Progress.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Cleaner data:** required information is captured before a record is saved, which supports accurate reporting and SLA tracking.
- **Immediate feedback:** users are told what is wrong at the moment they act, on the same screen.
- **Consistency:** Urgency follows Impact automatically and cannot be edited on high-impact incidents.
- **Adaptive behaviour:** Reverse if false means the form returns to normal when the condition no longer applies.
- **Protection of State:** quick edits from the list cannot bypass the form.
- **Lightweight and simple:** a small set of configurations and short scripts that are easy to implement and understand.

Disadvantages

- **Client-side only:** these rules run in the browser, so records created by other routes, such as imports or integrations, are not covered by them.
- **Urgency is not reset:** the onChange script sets Urgency when Impact becomes High but does not change it back when Impact is lowered; the field simply becomes editable again.
- **No exceptions for roles:** the onCellEdit script blocks every list edit of State, including for users who might legitimately need it.
- **Late warning for Assigned to:** the requirement is enforced when the user submits, so it is not flagged as mandatory while filling in the form.
- **Hard-coded values:** the impact and urgency values and message texts are written into the scripts, so changes require editing the scripts.

9. CONCLUSION

The Implement Client Script & UI Policy (Incident) project shows how UI Policies and Client Scripts can work together to shape the way an Incident form behaves and to keep incorrect data from being submitted. By combining a UI Policy and its actions with onChange, onSubmit and onCellEdit scripts, we made fields such as Assignment group, Assigned to and Urgency respond sensibly to changes in Impact, which keeps the data clean and consistent.

Testing confirmed each rule in practice: the save was blocked with a clear message when Assigned to was missing, Urgency followed a High impact, the fields returned to normal when Impact was lowered, and State could not be changed from the list but could be changed from the form. The solution is lightweight, efficient and

easy to implement, which makes it well suited to a short micro project while still reflecting good practice for improving form usability and data integrity in ServiceNow.

10. FUTURE SCOPE

There are several natural ways to build on this work:

- Reset Urgency to a suitable value when Impact is lowered, so the two fields stay aligned in both directions.
- Add role-based exceptions to the onCellEdit script so authorised users can still make list edits where appropriate.
- Show Assigned to as mandatory on the form itself for High impact, so users see the requirement before they submit.
- Reinforce the rules with server-side controls so that records created through imports or integrations follow the same standards.
- Extend the approach to other impact levels, other fields and other tables, using the same pattern of a UI Policy plus supporting Client Scripts.
- Move message texts and values into configurable messages or properties so they can be changed without editing scripts.

11. APPENDIX

Source Code

The client scripts below are those configured in the ServiceNow instance.

Client Script 1: Auto set urgency for high impact (onChange, field: Impact)

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue == '') {
    return;
  }
  if (newValue == '1') { // High Impact
    g_form.setValue('urgency', '1');
    g_form.showFieldMsg(
      'impact',
      'High impact incidents must have High urgency and an assignment group.',
      'info'
    );
  }
}
```

Client Script 2: Prevent save if assigned to missing (onSubmit)

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
    g_form.showErrorBox(
      'assigned_to',
      'Assigned To is mandatory for High impact incidents.'
    );
    return false;
  }
  return true;
}
```

Client Script 3: Prevent state change via list edit (onCellEdit, field: State)

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be changed from the list. Please open the Incident.');
```

```
// Cancel the list edit  
    callback(false, oldValues);  
}
```

Dataset Link

Not applicable. The project works on the standard Incident table in the ServiceNow instance, and no external dataset was used.

GitHub & Project Demo Link

Item	Link
GitHub repository	https://github.com/Rash6906/Implement-Client-Script-and-UI-Policy-Incident-.git
Project demo video	https://drive.google.com/file/d/1IsgcZhEpz95CWjBMqLH3ClnFkmdDAWJ/view?usp=sharing